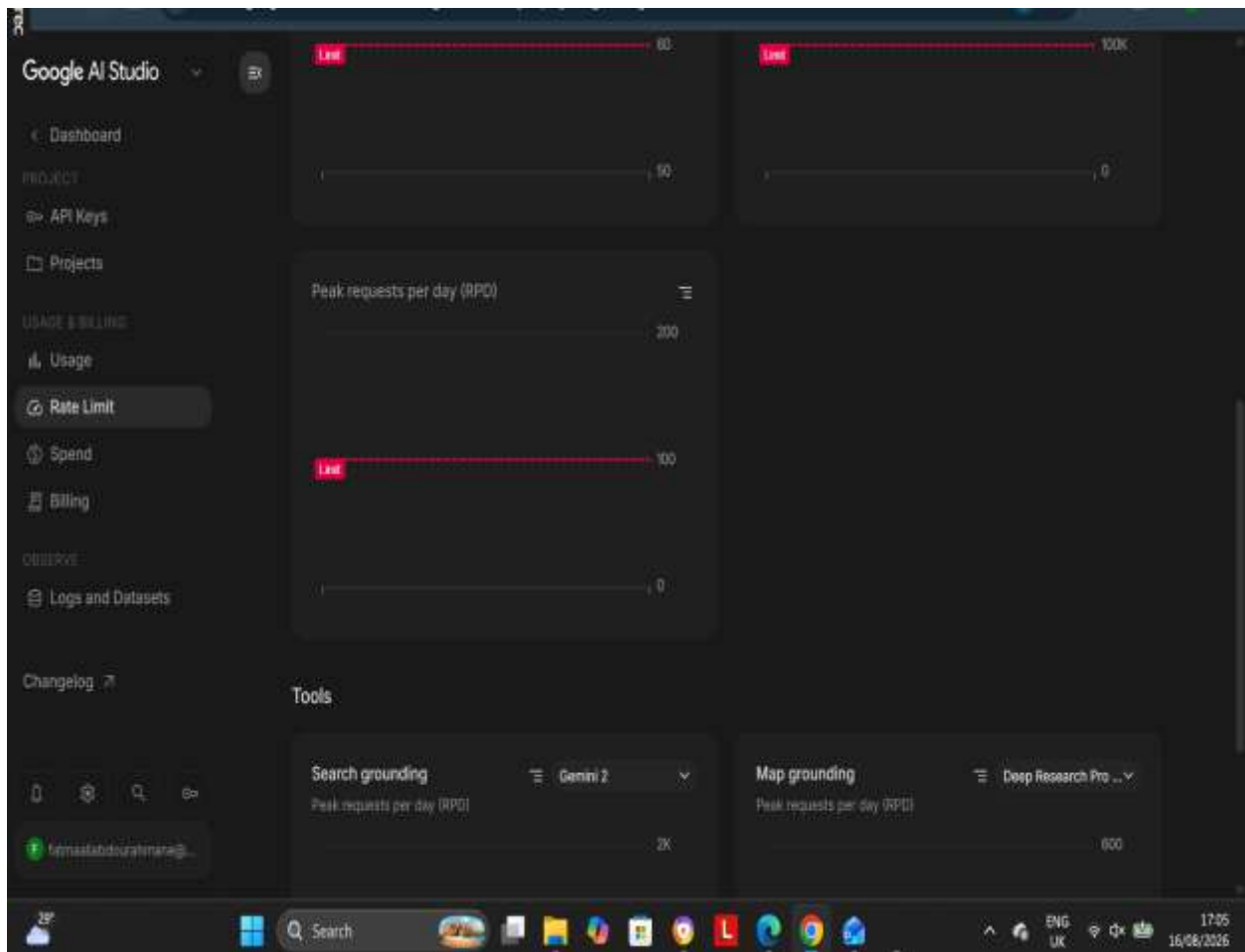




API Keys

Manage your project API keys. Remember to keep your API keys safe to prevent unauthorized access.

[+ Create API Key](#)

NAME	SECRET KEY	CREATED	LAST USED	EXPIRES	USAGE (24HRS)

Only team owners or users with the developer role may create or manage API keys.

Makes sense — that `CERTIFICATE_VERIFY_FAILED` error is a local VS Code/Windows networking issue (a corporate proxy or antivirus intercepting SSL), not something wrong with your code or Gemini. Moving to Colab should sidestep it entirely since Colab runs on Google's own network with proper certs.

Here's how to set it up cleanly in Colab with your Gemini key:

1. Install the SDK (Colab doesn't have it by default)

```
python
!pip install -q google-gemini
```

2. Store your key in Colab Secrets (never hard-code it)

Click the  key icon in the left sidebar → "Add new secret" → name it `GEMINI_API_KEY` → paste your key → toggle "Notebook access" on.

3. Read the key and create the client

```
python
from google.colab import userdata
GEMINI_API_KEY = userdata.get('GEMINI_API_KEY')
```

Ecrire un message...

no now that it is in colab how do i put it in the folder of the lab the folder contains gitignore and .env

Here's the workflow to get your Colab notebook into that local project folder alongside your `.gitignore` and `.env`:

Option A: Download from Colab, then move it locally (simplest)

1. In Colab: File → Download → Download .ipynb
2. This saves it to your computer's Downloads folder.
3. Move/replace it into your project folder:
 - Windows (PowerShell):

```
powershell
move "$env:USERPROFILE\Downloads\lab_4.ipynb" "C:\path\to\lab-4-llm-decision-su
```

- Or just drag-and-drop it in File Explorer.

Ecrire un message...

I added it now how to push from colab because I have already pushed the files since they were opened in vs code

A réfléchi pendant 5 s

Since the repo already exists on GitHub (from your VS Code pushes), here's how to push your Colab changes into it:

Easiest: Colab's built-in GitHub save

1. In Colab: **File** → **Save a copy in GitHub**
2. Authorize GitHub access if it's your first time (opens a popup to grant Colab access to your account).
3. In the dialog:
 - Repository: select `lab-4-llm-decision-support` (your existing repo)
 - Branch: `main` (or whatever your default branch is)
 - File path: make sure this matches the **exact same path** the notebook already has in the repo (e.g. `lab_4.ipynb` at root — not a new path, or it'll create a duplicate file instead of updating the existing one)
4. Write a commit message e.g. "Colab: Gemini setup + Part 1 API calls"

Écrire un message...



Sonnet 5 Moyen



ClientError
call last)

Traceback (most recent

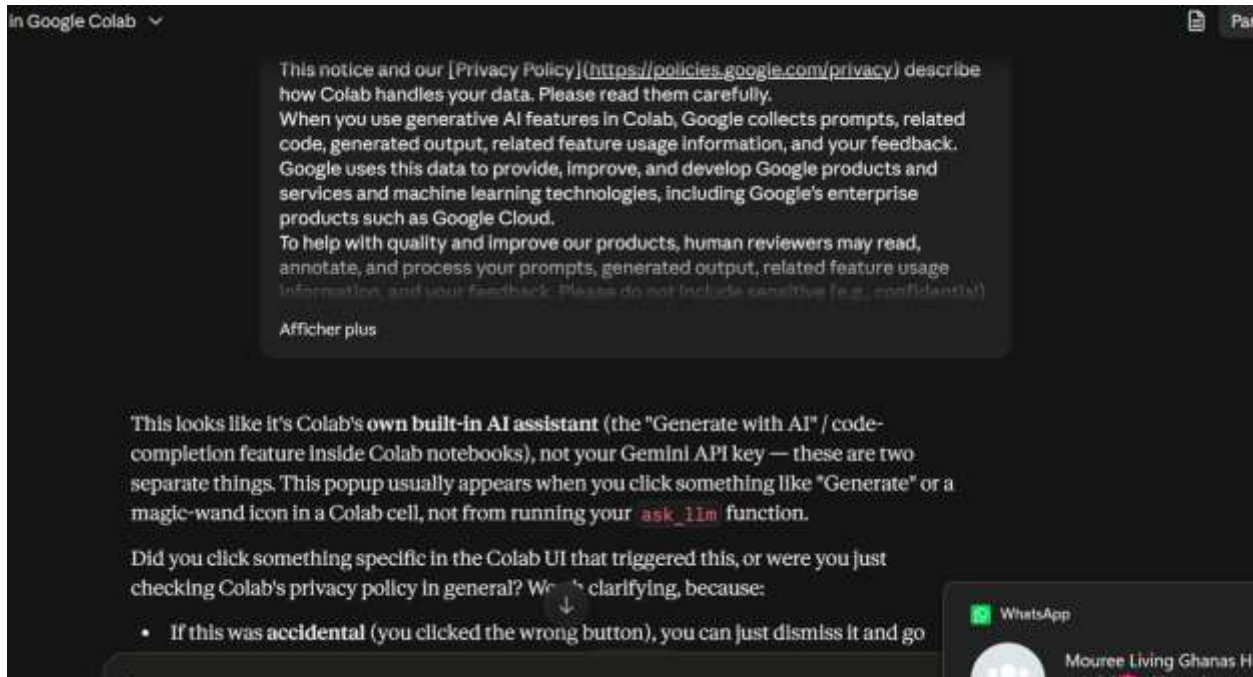
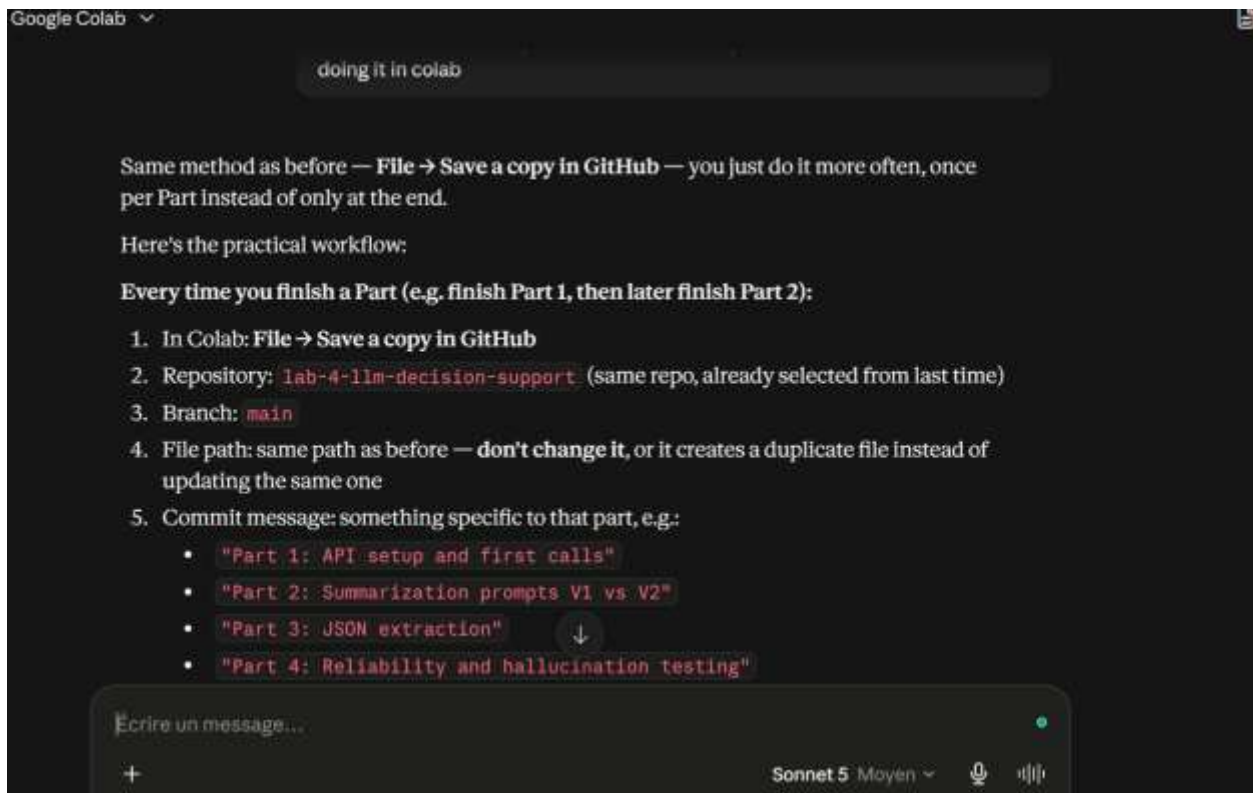
```
/tmp/ipykernel_2492/93528835A.py in <cell line: 8>():
```

Afficher plus

Web recherché

Good catch by the error itself — turns out I steered you wrong earlier. As of now (August 2026), `gemini-2.5-flash` is being phased out and blocked for new users Gemini 2.5 Flash will shut down on October 16, 2026, and does not require the pay-as-you-go plan if using the Gemini Developer API, which lines up with the 404 you're seeing. [Firebase](#)

Your original notebook actually had the right idea — Gemini's current generation is 3.x, and Gemini 3.6 Flash launched July 21, 2026 and is where most teams should start, built for coding, knowledge work, and multimodal tasks. [Second Talent](#)



but what is the problem

The problem is simple: you've used up your Gemini API's **free daily limit of 20 requests** for the `gemini-3.6-flash` model. That popup about Colab's AI privacy policy is unrelated — different feature, ignore it for now.

So right now, if you try to call `ask_llm(...)` again, it will fail with the same `429 RESOURCE_EXHAUSTED` error, because Google is blocking any more calls to that model until your quota resets.

What this means practically:

- You can't run any more Gemini API calls today (or until the daily reset).
- This isn't a bug in your code — your code is working fine. It's just a usage cap on the free tier.

What to do:

1. Check your actual quota status here: <https://ai.dev/rate-limit> — this will tell you exactly when it resets and how many you've used.
2. If you need to keep working today, look at whether a different Gemini model (like a "lite"

Écrire un message...



A réfléchi pendant 4 s

These two screenshots are actually showing a **different model** — the dropdown says "Antigravity" (top right of image 1, under "Peak usage trends"), not `gemini-3.6-flash`, which is the model your error message specifically named. So this page isn't showing you the limit that's actually blocking you.

Here's what to do:

1. On the **Peak usage trends** section (image 1), click the model dropdown that currently says "Antigravity"
2. Select `gemini-3.6-flash` from that list instead
3. That will update the "Peak requests per day (RPD)" chart (bottom of image 2) to show your actual usage/limit for the model you're using — you should see it maxed out at 20/20 or close to it, matching your error message

Écrire un message...

```
Attempt 1 failed (503 UNAVAILABLE. {'error': {'code': 503,
'message': 'This model is currently experiencing high demand.
Spikes in demand are usually temporary. Please try again later.',
'status': 'UNAVAILABLE'}}). Retrying in 15s...
Attempt 2 failed (503 UNAVAILABLE. {'error': {'code': 503,
'message': 'This model is currently experiencing high demand.
Spikes in demand are usually temporary. Please try again later.',
'status': 'UNAVAILABLE'}}). Retrying in 15s...
```

Afficher plus

`gemini-flash-latest` is genuinely down/overloaded right now — 3 retries with 15s gaps all failed the same way, so this isn't temporary flakiness you can wait out quickly. Given your deadline, switch to **Groq** now — it's reliable, free, and was literally listed as the recommended option in your own lab instructions.

1. Get a free Groq key (~1 minute)

Go to <https://console.groq.com/keys> → sign in → create API Key" → copy it.

2. Add it to Colab Secrets

Écrire un message...



Sonnet 5 Moyen



for first cell

```
-----
-----
```

```
SecretNotFoundError
call last)
```

Traceback (most recent

Afficher plus

This just means the secret wasn't actually saved yet in Colab — the code is fine, you just need to go add it.

Steps:

1. Look at the **left sidebar** of Colab — click the  (key icon).
2. Click "+ Add new secret"
3. In the **Name** field, type exactly: `GROQ_API_KEY` (capital letters, exact spelling matters)

provide youtube link for tutorials to help do this lab

Web recherché

Here are a few solid tutorials, though given your lab is essentially already done, these are more for future reference:

Gemini API + Python basics:

- How to Use the Gemini API in Python in 2026 — covers sending requests, adding system instructions, and getting started with the Google Gen AI Python SDK
https://www.youtube.com/watch?v=3CW_mr00xYM YouTube

Gemini + Google Colab specifically:

- How To Use Gemini In Google Colab (Step-by-Step) — shows how to integrate Gemini AI inside a Colab notebook
<https://www.youtube.com/watch?v=VBckB2SEZPI>

Écrire un message...



Sonnet 5 Moyen



System prompts, roles, and API calls (matches your Section 1-2 work):

https://www.youtube.com/watch?v=3CW_mr00xYM

Covers sending requests, system instructions, and the basic anatomy of a call — exactly what your first TODOs were about.

Prompt engineering fundamentals — zero-shot vs few-shot, temperature (matches Section 3):

Search "prompt engineering crash course few-shot zero-shot temperature" on YouTube — there are several good ones from channels like freeCodeCamp and IBM Technology that cover exactly the V1-vs-V2 prompting and temperature concepts you worked through.

Structured JSON extraction from LLMs (matches your Section 3.2 extraction work):

The written guides I found are actually excellent and directly match what you built — this one is worth reading even without video: <https://www.learnwithparam.com/blog/prompt-engineering-structured-json-output> — it walks through few-shot examples and output priming to force clean JSON, which is exactly your `extract_fields` approach.

Given you're likely submitting very soon, I'd focus energy on finishing the lab rather than watching tutorials right now — but bookmark these for after submission, since they'll help the concepts stick for future labs. Want to go back to finishing the wrap-up steps (name/ID, final commit) instead?

Écrire un message...